PFA - Backend Project Architecture & Style Guide

Table of Contents

Project Architecture Overview 01
Logical Grouping 01
Application 02
Controllers (API Layer) 03
Services (Business Logic Layer) 05
Repositories (Data Access Layer) 12
Models (Entities) 14
Security Components 19
Authentication Flow (JWT Filter) 21
DTOs and Request Objects 22
Configuration (App Level)..... 26
Schedulers 30
Resources 31
Testing 32

Project Architecture Overview

This project follows a layered Spring Boot architecture, separating responsibilities into distinct components to improve maintainability, scalability, and clarity.

High-Level Flow: Client → Controller → Service → Repository → Database
↓
DTOs

Logical Grouping

Standard Spring Boot package structure:

- com.project.config: Security, Password Encoders, Flyway configs.
- com.project.controllers: All REST Endpoints.
- com.project.services: Business logic classes.
- com.project.repositories: JPA Interfaces.
- com.project.models: Database Entities (The tables).
- com.project.dto:
    - /request: Objects coming in (e.g., UserRegistrationRequest).
    - /response: Objects going out (e.g., UserProfileDTO).
- com.project.security: UserPrincipals and authority handling.
- com.project.schedulers: Async tasks and web-scraping triggers.

Application Layer

Purpose: Entry point of the application.

- Initializes Spring Boot
- Configures application-wide settings
- Enables auto-routing for all endpoints
- Connects all components together

File: `PfaApplication.java`

Layer: Application
Purpose: Entry point of the backend application responsible for bootstrapping and starting the Spring Boot system.

Initializes the application context, enables core features, and starts the embedded web server.

Responsibilities:

- Starts the Spring Boot application
- Initializes all components (controllers, services, repositories)
- Enables scheduling and asynchronous processing
- Configures auto-detection of application components

Key Features:

- Uses `@SpringBootApplication` for auto-configuration and component scanning
- Enables scheduled tasks with `@EnableScheduling`
- Supports asynchronous execution with `@EnableAsync`
- Launches embedded server (e.g., Tomcat)

Controllers (API Layer)

Purpose: Handle HTTP requests and responses.

- Define API endpoints using `@RequestMapping`
- Accept incoming data (Request Objects)
- Call Service layer for business logic
- Return `ResponseEntity` (with HTTP status codes + data)

File: `AdminController.java`

Purpose: Provides admin-only endpoints for managing users.

Responsibilities:

- Retrieve all users in the system
- Promote users to ADMIN role

Endpoints:

- `GET /api/admin/getAll` → Returns all users + timestamp
- `PATCH /api/admin/{id}/promote` → Promotes a user to ADMIN

Key Features:

- Restricted to ADMIN role (handled by security configuration)
- Uses `ResponseEntity` for proper HTTP status handling

Note: This controller enforces role-based access control for administrative actions.

File: `PetController.java`

Purpose: Handles all pet-related API operations, including retrieval and web scraping.

Responsibilities:

- Fetch all pets from database
- Retrieve pet by ID
- Trigger external web scraping for pet data

Endpoints:

- `GET /api/pets/getAll` → Returns all pets + timestamp
- `GET /api/pets/{id}` → Returns a specific pet
- `GET /api/pets/scrape` → Runs web scraper and updates database

Key Features:

- Integrates with `WebScraperService` for external data

- Uses `DatabaseBackupService` before scraping (data safety)
- Synchronizes scraped data with database via `PetService`

Note: This controller connects external data sources → backend → database, making it a key integration point.

File: `PingController.java`

Purpose: Provides a simple health-check endpoint to verify that the API is running.

Responsibilities:

- Respond to basic GET request
- Return server status and timestamp

Endpoint:

- `GET /api/public/ping`

Key Features:

- Public endpoint (no authentication required)
- Lightweight and fast

Note: Useful for: Testing API availability and Monitoring system health

Services (Business Logic Layer)

Service Layer Flow

```
Controller → Service → Repository → Database
            ↓
        Security / JWT / Backup Logic
```

Purpose: Core logic of the application.

- Processes data
- Applies rules (validation, authentication, tokens, etc.)
- Communicates with repositories

Example responsibilities:

- User authentication
- Token generation
- Data validation

Note: If it involves logic or decision-making, it belongs in Services.

File: `CustomUserDetailsService.java`

Purpose: Acts as the bridge between the application's user database and Spring Security's authentication system.
This service is required for login to work, it allows Spring Security to retrieve user credentials and roles.

Responsibilities

- Load user data from the database during authentication
- Normalize email input (trim + lowercase)
- Convert `User` entity into `UserPrincipal` (Spring Security format)

Dependencies

- `UserRepository`
- `UserPrincipal`

Key Features

- Implements `UserDetailsService` (required by Spring Security)
- Throws `UsernameNotFoundException` if user does not exist
- Ensures consistent email lookup (case-insensitive)

File: `DatabaseBackupService.java`

Purpose: Handles database backup operations by creating compressed SQL dump files. Provides data safety and recovery support, especially before operations like web scraping or bulk updates.

Responsibilities

- Generate timestamped database backups
- Execute system-level backup command (`mysqldump`)
- Log success or failure of backup process

Dependencies

- Application configuration (`application.properties`)
- MySQL database

Key Features

- Automatically builds backup filenames using timestamps
- Compresses backups using `gzip`
- Uses environment-configured database credentials

Note: This service uses a system command (`ProcessBuilder`), which means:

- It depends on the server environment
- `mysqldump` must be installed and accessible

File: `JWTService.java`

Purpose: Manages JSON Web Token (JWT) creation, parsing, and validation for authentication.
This is the core authentication engine, it enables secure, stateless login across the API.

Responsibilities

- Generate JWT tokens for authenticated users
- Extract user information (email) from tokens
- Validate token authenticity and expiration

Dependencies

- Spring Security (`UserDetails`)
- JJWT library

Key Features:

- **Token Generation**
  - Includes:
    - Subject (email)
    - Issue time
    - Expiration (30 minutes)
- **Token Validation**
  - Confirms:
    - Token matches user
    - Token is not expired
- **Claim Extraction**
  - Extracts:
    - Email (subject)
    - Expiration date
- **Stateless Authentication Support**
  - No sessions required
  - All authentication handled via tokens

File: `PetService.java`

Purpose: Handles all business logic related to pets, including retrieval, transformation, and synchronization of scraped data with the database.

Responsibilities

- Retrieve pet data from the database
- Convert `Pet` entities into `PetDTO` objects
- Synchronize scraped pet data with existing database records
- Maintain data consistency (add, update, deactivate pets)

Dependencies

- `PetRepository`
- `AdoptionSiteRepository`
- `PetDTO`

Core Methods:

`findAll()`

- Returns all pets as `PetDTO`
- Prevents exposing internal entity structure

`findById(id)`

- Retrieves a single pet
- Throws exception if not found

`findByName(name)`

- Retrieves pets by name
- Returns list of DTOs

`sync(scrapedPets)`

Purpose:

Compares scraped pet data with database records and applies changes.

Sync Logic Flow

Scraped Data → Compare with DB →
→ Add new pets
→ Update changed pets
→ Deactivate missing pets

Key Operations

- **Add:** If pet exists in scrape but not DB → save new pet
- **Update:** If pet exists but fields changed → update fields
- **Deactivate:** If pet exists in DB but not in scrape → mark as `INACTIVE`

Key Features:

➢ **Smart Comparison System**
  - Uses a composite key (`buildKey()`) to uniquely identify pets
  - Prevents duplicates across scraping runs

➢ **Change Detection**
  - Only updates when important fields change:
    - location
    - price
    - Status

➢ **Data Integrity**
  - Ensures DB always reflects latest scraped data

Note: Instead of blindly overwriting data, this service performs a diff-based sync, which:

- Reduces unnecessary database writes
- Preserves stable data
- Improves performance

Includes an experimental `syncBySite()` method for site-specific synchronization, showing scalability for multi-source data.

Pet Data Flow (Scraping)

Scheduler → WebScraperService → PetService.sync()
→ Compare DB vs Scraped
→ Add / Update / Deactivate

File: `UserService.java`

Purpose: Handles all user-related business logic, including authentication, registration, and role management.

Responsibilities

- Manage user data (CRUD operations)
- Handle authentication (login)
- Generate JWT tokens
- Enforce security practices (password hashing)
- Manage user roles (e.g., ADMIN promotion)

Dependencies

- `UserRepository`
- `PasswordEncoder`
- `AuthenticationManager`
- `JWTService`

Core Methods:

`findAll()`

- Returns all users as `UserDTO`

`findById(id)`

- Retrieves user by ID
- Converts to DTO

`findByEmail(email)`

- Retrieves user by email
- Used during authentication flows

`findByJWT(request)`

Purpose: Extracts user identity from JWT in request header.

Flow: Request → Extract JWT → Extract Email → Query DB → Return UserDTO

Security Insight:

If this fails, it may indicate:

- Expired token
- Invalid token
- Potential malicious request

`createUser(request)`

Purpose: Registers a new user.
Steps

1. Normalize email (trim + lowercase)
2. Hash password using `PasswordEncoder`
3. Assign default role (`ROLE_USER`)
4. Save user to database

Note: Ensures **secure storage of user credentials**

`verify(userRequest)`

Purpose: Authenticates users and generates JWT.
Flow: Login Request → AuthenticationManager →
→ Valid → Generate JWT
→ Invalid → Reject

Key Features

- Uses Spring Security authentication pipeline
- Only generates token if authentication succeeds
- Returns JWT for stateless authentication

Note: This is the core login mechanism of your system.

`promoteToAdmin(userId)`

Purpose: Upgrades a user's role to ADMIN.
Key Features

- Runs inside `@Transactional`
- Automatically persists role change

Note: Supports role-based access control (RBAC) in your application.

File: WebScraperService.java

Layer: Service
Purpose: Scrapes pet adoption websites by crawling pages, extracting pet profile links, gathering pet data (name, breed, age, etc.), and converting it into `Pet` entities.

Handles both static and dynamic websites using Selenium and Jsoup, including complex components like Petfinder widgets.

Responsibilities:

- Crawls websites using a BFS (Breadth-First Search) approach
- Identifies supported providers (ShelterLuv, Petfinder)

- Extracts pet profile URLs from pages and iframes
- Scrapes detailed pet data from profile pages
- Converts raw scraped data into `Pet` entities using `ScrapedPetDTO`

Key Features:

- Uses Selenium WebDriver for JavaScript-rendered pages
- Uses Jsoup for HTML parsing
- Supports Shadow DOM interaction (Petfinder pet-scroller)
- Implements Builder Pattern for structured data extraction
- Handles pagination within embedded widgets
- Filters and validates scraped data before conversion

Repositories (Data Access Layer)

Purpose: Communicate with the database.

- Interface layer using JPA
- Handles CRUD operations
- No business logic

Uses Spring Data JPA, so many methods are auto-generated (e.g., `findById`, `save`, `delete`**)**

Note: All database access must go through repositories.

File: `AdoptionSiteRepository.java`

Layer: Data Access

Purpose: Provides database access operations for AdoptionSite entities. Acts as the interface between the application and the database for adoption site data.

- Responsibilities
- Retrieve adoption sites from the database
- Support searching by site name
- Fetch adoption site by unique site ID
- Leverage Spring Data JPA for automatic query generation

Dependencies

- AdoptionSite (Entity)
- Spring Data JPA (JpaRepository)

Key Features

- Extends JpaRepository to inherit built-in CRUD operations
- Uses method naming conventions to auto-generate queries
- Returns Optional for safe handling of nullable results
- Supports custom query methods without manual SQL

File: `PetRepository.java`

Layer: Data Access

Purpose: Handles all database operations related to Pet entities. Enables retrieval and filtering of pet data based on different criteria.

Responsibilities

- Retrieve pet records from the database
- Search pets by name

- Filter pets based on associated adoption site
- Provide CRUD operations through JpaRepository

Dependencies

- Pet (Entity)
- Spring Data JPA (JpaRepository)

Key Features

- Extends JpaRepository for built-in CRUD functionality
- Uses derived query methods (findByName, findBySite_SiteId)
- Supports relational queries through entity mapping
- Eliminates need for manual SQL queries

File: `UserRepository.java`

Layer: Data Access
Purpose: Provides database access operations for User entities. Supports user lookup and authentication-related queries.

Responsibilities

- Retrieve users from the database
- Search users by name
- Find user by email for authentication
- Leverage Spring Data JPA for CRUD operations

Dependencies

- User (Entity)
- Spring Data JPA (JpaRepository)

Key Features

- Extends JpaRepository to inherit built-in CRUD methods
- Uses derived query methods (`findByName, findByEmail`)
- Returns Optional for safe null handling
- Supports authentication by enabling email-based lookup

Models (Entities)

Purpose: Represent database tables.

- Each model = one table
- Contains fields mapped to columns
- Used internally in backend

➢ **BaseModel**

Purpose: Shared properties for all models.

- Common fields (e.g., `id`, timestamps)
- Extended by other models

➢ **Example: User Model**

- Represents user data
- Can be manipulated internally
- NOT safe to expose directly (contains sensitive info like passwords)

File: `AdoptionSite.java`

Layer: Model
Purpose: Represents an adoption site stored in the database. Defines the structure and schema for adoption site data.

Responsibilities

- Store adoption site information (name, contact info, rating, location)
- Map object fields to database columns
- Serve as a persistent entity used by repositories

Dependencies

- Jakarta Persistence (JPA Annotations)
- Lombok (@Data)

Key Features

- Annotated with `@Entity` for database mapping
- Uses `@Id` and `@GeneratedValue` for primary key
- Includes column constraints (nullable, naming)
- Uses Lombok to reduce boilerplate code
- Supports both default and parameterized constructors

File: `Comments.java`

Layer: Model
Purpose: Represents user comments stored in the database. Links user-generated feedback to specific users.

Responsibilities
- Store comment text and date
- Associate comments with users
- Define database schema for comments

Dependencies
- User (Entity)
- Jakarta Persistence (JPA Annotations)
- Lombok (@Data)

Key Features
- Uses @Entity for persistence mapping
- Implements Many-to-One relationship with User
- Uses lazy loading for performance optimization
- Stores timestamp using LocalDate
- Includes required and optional field constraints

File: `FeaturedPets.java`

Layer: Model
Purpose: Represents pets that are highlighted or featured within the system. Stores metadata about when and why a pet is featured.

Responsibilities
- Track featured pet duration (start and end dates)
- Associate a pet with its featured status
- Store reason for featuring a pet
- Define database schema for featured pets

Dependencies
- Pet (Entity)
- Jakarta Persistence (JPA Annotations)
- Lombok (@Data)

Key Features
- Uses @Entity for persistence mapping
- Implements One-to-One relationship with Pet
- Stores feature time window (startDate, endDate)

- Includes reason field for administrative context
- Uses lazy loading for performance optimization

File: `Pet.java`

Layer: Model
Purpose: Represents a pet stored in the database. Serves as the core entity for pet-related data across the application.

Responsibilities
- Store pet details (name, breed, age, gender)
- Store classification and metadata (type, location, status)
- Associate pets with adoption sites
- Maintain compatibility scoring for users

Dependencies
- AdoptionSite (Entity)
- Jakarta Persistence (JPA Annotations)
- Lombok (@Data)

Key Features
- Annotated with @Entity for database persistence
- Uses @ManyToOne relationship to link AdoptionSite
- Includes unique constraint to prevent duplicate pets
- Stores both descriptive and operational data (status, price)
- Supports compatibility scoring for matching logic

File: `PetRating.java`

Layer: Model
Purpose: Represents user-submitted ratings for pets. Tracks how users evaluate pets within the system.

Responsibilities
- Store rating values associated with pets
- Track which user submitted a rating
- Maintain timestamp of when rating was created
- Define schema for rating relationships

Dependencies
- User (Entity)
- Pet (Entity)
- Jakarta Persistence (JPA Annotations)

- Lombok (@Data)

Key Features

- Uses @Entity for persistence mapping
- Implements Many-to-One relationships with User and Pet
- Stores rating value and date
- Supports user-driven feedback system
- Uses lazy loading for relational efficiency

File: `Reviews.java`

Layer: Model
Purpose: Represents user-submitted reviews for adoption sites. Stores ratings and feedback associated with specific users and sites.

Responsibilities

- Store review ratings and comments
- Associate reviews with users and adoption sites
- Track when reviews are submitted
- Define schema for user feedback

Dependencies

- User (Entity)
- AdoptionSite (Entity)
- Jakarta Persistence (JPA Annotations)
- Lombok (@Data)

Key Features

- Uses @Entity for database persistence
- Implements Many-to-One relationships with User and AdoptionSite
- Stores rating, comment, and review data
- Supports user-generated feedback system
- Maintains relational integrity between users and sites

File: `Submissions.java`

Layer: Model
Purpose: Represents user-submitted pet entries for review or approval. Allows contributors to submit pet information into the system.

Responsibilities

- Store submitted pet information
- Track submission date

- Associate submissions with users
- Maintain submission status

Dependencies

- User (Entity)
- Jakarta Persistence (JPA Annotations)
- Lombok (@Data)

Key Features

- Uses @Entity for persistence mapping
- Implements Many-to-One relationship with User (contributor)
- Stores raw pet information as text
- Tracks submission lifecycle using status field
- Supports user-generated content workflow

File: `User.java`

Layer: Model
Purpose: Represents a user within the system. Stores authentication credentials and authorization roles.

Responsibilities

- Store user identity information (name, email)
- Manage authentication credentials (hashed password)
- Define user roles for authorization
- Persist optional metadata (location)

Dependencies

- Jakarta Persistence (JPA Annotations)
- Lombok (@Data)

Key Features

- Annotated with `@Entity` for database persistence
- Uses `@Enumerated(EnumType.STRING)` for role storage
- Includes Role enum for authorization control
- Stores password securely (intended for hashed values)
- Supports role-based access control (USER vs ADMIN)

Security Components

UserPrincipal

Purpose: Spring Security-managed user object

- **Stores:**
  - Roles/authorities
  - Email
  - Hashed password
- **Used for authentication & authorization**

File: `CustomAccessDeniedHandler.java`

Layer: Security

Purpose: Handles 403 Forbidden errors when a user is authenticated but does not have permission to access a resource.

Improves API consistency by ensuring all errors return clean JSON responses instead of default HTML errors.

Responsibilities:

- Intercepts access denied exceptions
- Returns a structured JSON response instead of default error page

Key Features:

- Sends HTTP status `403 FORBIDDEN`
- Returns JSON response: `error and Message`

File: `CustomAuthenticationEntryPoint.java`

Layer: Security

Purpose: Handles 401 Unauthorized errors when a user is not authenticated.

Ensures unauthenticated requests are handled consistently across the API.

Responsibilities:

- Intercepts authentication failures
- Returns a JSON response when authentication is missing or invalid

Key Features:

- Sends HTTP status `401 UNAUTHORIZED`
- Returns structured JSON error message

File: `JwtFilter.java`

Layer: Security / Authentication
Purpose: Authenticates incoming requests using JSON Web Tokens (JWT).

This is the core of authentication, without it, protected endpoints would not work.

Responsibilities

- Extracts JWT from `Authorization` header
- Validates token using `JWTService`
- Loads user via `CustomUserDetailsService`
- Sets authentication in `SecurityContext`

Request Flow

Request → JwtFilter → Validate Token → Set SecurityContext → Continue

Key Features

- Runs once per request (`OncePerRequestFilter`)
- Enables stateless authentication (no sessions)
- Rejects invalid or malformed tokens immediately (401)

Authentication Flow

Login Request → UserController → UserService.verify()
→ AuthenticationManager
→ CustomUserDetailsService
→ UserRepository
→ JWTService.generateToken()
→ Return Token

File: `UserPrincipal.java`

Layer: Security Component
Purpose: Acts as an adapter between the User entity and Spring Security. Allows the application to integrate user data into the authentication system.

Responsibilities
- Provide user credentials to Spring Security
- Convert user roles into granted authorities
- Define account status (locked, expired, enabled)
- Serve as the authenticated user representation

Dependencies
- User (Entity)
- Spring Security (UserDetails, GrantedAuthority)

Key Features
- Implements UserDetails (required by Spring Security)
- Maps user roles to GrantedAuthority format
- Uses email as the username for authentication
- Wraps User entity without exposing it directly
- Ensures compatibility with authentication framework

DTOs and Request Objects

Used to transfer data between layers and control what information is exposed in API responses.

DTOs (Data Transfer Objects)

Purpose: Data sent OUT of the API.

- Used to send safe, structured data to clients
- Excludes sensitive information (e.g., passwords)

Request Objects

Purpose: Data coming INTO the API.

- Used for deserializing incoming JSON
- Represents client input

Example: `UserRequest` → used when creating an account

DTO vs Request

Type	Direction	Purpose
Request Object	Incoming	What client sends
DTO	Outgoing	What API returns

- **Request Objects (`Incoming`):** These are specifically for deserializing JSON from the client. They should include validation (e.g., "Email must not be null").
- **DTOs (`Outgoing`):** These are for packaging data. We use these to hide passwords, internal IDs, or "CreatedBy" timestamps that the front-end doesn't need.

File: `UserController.java`

Purpose: Manages user-related operations including registration, authentication, and retrieval.

Responsibilities

- Register new users
- Authenticate users (login)
- Retrieve user data
- Resolve currently authenticated user via JWT

Endpoints

- `GET /api/users/{id}` → Get user by ID
- `GET /api/users/findMe` → Get current authenticated user
- `POST /api/users/register` → Create new user
- `POST /api/users/login` → Authenticate user and return JWT

Key Features:

- **JWT-Based Authentication**
  - Generates token on login/register
  - Used for protected endpoints
- **Secure Data Handling**
  - Returns `UserDTO` instead of full `User`
  - Prevents sensitive data exposure
- **Validation**
  - Uses `@Valid` for request validation
- **Error Handling**
  - Returns proper HTTP statuses (201, 200, 403)

Note: This is the core user interaction controller and central to authentication flow.

File: `PetDTO.java`

Layer: DTO
Purpose: Represents a simplified and controlled version of Pet data for transfer between backend and client. Prevents exposing full entity details and improves API response structure.

Responsibilities

- Encapsulate pet data for API responses
- Provide a clean structure for frontend consumption
- Convert Pet entities into DTO format

Dependencies

- Pet (Entity)

Key Features

- Uses Java record for concise and immutable data structure
- Includes only necessary fields for client use
- Provides static factory method (fromEntity) for conversion
- Improves security by limiting exposed data

File: `ScrapedPetDTO.java`

Layer: DTO
Purpose: Represents pet data obtained from external web scraping sources. Transforms unstructured scraped data into a structured format usable by the application.

Responsibilities

- Map raw scraped data into a structured DTO
- Convert external data into internal Pet entity format
- Handle missing or null values safely
- Provide default values for incomplete scraped data

Dependencies

- Pet (Entity)
- Jackson ObjectMapper

Key Features

- Uses @JsonProperty to map external JSON field names
- Includes `fromMap()` for converting raw data into DTO
- Provides `toEntity()` to transform DTO into database-ready entity
- Applies default values to prevent null-related errors
- Acts as a bridge between external data sources and internal models

File: `UserDTO.java`

Layer: DTO
Purpose: Represents a safe and controlled version of User data for API responses. Prevents exposure of sensitive information such as passwords.

Responsibilities

- Expose only safe user data to clients
- Convert User entities into DTO format
- Maintain a clean separation between persistence and API layers

Dependencies

User (Entity)

Key Features

- Uses Java record for concise and immutable structure
- Excludes sensitive fields (e.g., password)
- Provides static factory method (fromEntity)
- Includes role information for authorization context
- Ensures secure API responses

Configuration

Purpose: Customize Spring Boot behavior.
Includes:
- Security configuration (public vs protected endpoints)
- Password encryption rules
- CORS (allowed origins)

Example:
- Define which endpoints require authentication
- Set password requirements

The application uses environment-specific configuration profiles (development, testing, and production) to separate concerns and ensure flexibility. This approach allows the system to adapt behavior, such as logging, database connections, and security settings, without modifying application code.

File: `SecurityConfig.java`

Layer: Configuration / Security
Purpose: Central configuration for application security rules and authentication behavior. This file controls the entire security behavior of the API, authentication, authorization, and request filtering.

Responsibilities
- Defines which endpoints are public vs protected
- Configures JWT-based authentication
- Sets password encryption strategy
- Manages CORS policy
- Registers security filters

Key Features
- ➢ **Stateless Authentication**
  - Uses JWT instead of sessions
  - `SessionCreationPolicy.STATELESS`
- ➢ **Endpoint Authorization**
  - Public:
    - `/api/users/login`
    - `/api/users/register`
    - `/api/public/**`
  - Admin-only:

        - `/api/admin/**`
    - All others require authentication
- **Password Security**
    - Uses `BCryptPasswordEncoder`
- **JWT Integration**
    - Adds `JwtFilter` before default authentication filter
- **CORS Configuration**
    - Allows:
        - `http://localhost:3000`
        - `https://adoptpetsforall.com`

File: `application-dev.properties`

Layer: Configuration – Development Environment

Purpose: Defines environment-specific settings for the development profile, including database connectivity, ORM behavior, migration tooling, and logging configuration.

Responsibilities:

- Configures MySQL database connection using environment variables
- Controls JPA and Hibernate behavior for schema validation
- Enables and configures Flyway database migrations
- Defines logging levels for debugging and monitoring
- Provides default credentials for development security configuration

Key Features:

- Uses environment variables (`SPRING_DATASOURCE_USERNAME`, `SPRING_DATASOURCE_PASSWORD`) for secure credential management
- Sets `ddl-auto=validate` to ensure schema consistency without modifying the database
- Enables SQL query logging for debugging (`show-sql=true`)
- Configures Flyway for automatic database migration on startup
- Includes retry logic for database connection (`spring.flyway.connect-retries`)
- Provides optional debug-level logging for Hibernate SQL output
- Includes default in-memory authentication credentials for development/testing

File: `application.properties`

Layer: Configuration – Base (Environment-Agnostic)

Purpose: Defines shared configuration settings applied across all environments, serving as the baseline for development, testing, and production profiles.

Responsibilities:

- Provides default ORM behavior
- Establishes consistent naming conventions for database tables and columns
- Defines shared settings inherited by environment-specific configurations

Key Features:

- Sets `ddl-auto=validate` to enforce schema consistency
- Enables SQL logging by default for visibility
- Applies `CamelCaseToUnderscoresNamingStrategy` for standardized database naming
- Acts as a fallback configuration for all active profiles

File: `application-test.properties`

Layer: Configuration – Testing / CI Environment

Purpose: Defines configuration for automated testing environments, ensuring behavior closely mirrors production while remaining isolated and repeatable.

Responsibilities:

- Configures database connection for test environment
- Aligns ORM behavior with production settings
- Enables and manages database migrations during tests
- Supports reliable execution in CI/CD pipelines

Key Features:

- Uses dedicated test database (`PFA_DB_TEST`)
- Sets `ddl-auto=validate` to match production schema enforcement
- Disables SQL logging for cleaner test output
- Enables Flyway migrations with controlled configuration
- Uses `baseline-on-migrate` to support pre-existing schemas

- Validates migrations before execution (`validate-on-migrate`)
- Provides logging for Flyway operations to aid debugging

File: `application-prod.properties`

Layer: Configuration – Production Environment

Purpose: Defines secure and optimized settings for the production environment, focusing on stability, performance, and security.

Responsibilities:

- Configures database connection using environment variables
- Disables unnecessary logging for performance
- Ensures strict schema validation
- Enables production-safe database migration behavior

Key Features:

- Uses environment variables for sensitive credentials
- Disables SQL logging (`show-sql=false`) for performance and security
- Enforces schema validation (`ddl-auto=validate`)
- Enables Flyway for controlled database migrations
- Configures MySQL connection property (`allowPublicKeyRetrieval=true`) for compatibility

Schedulers

Purpose: Run background tasks.

- Executes methods at scheduled times
- Runs asynchronously (outside main thread)

Example:

- Web scraping jobs
- Periodic cleanup tasks

File: `WebScraperScheduler.java`

Layer: Scheduler / Background Tasks
Purpose: Handles scheduled background jobs related to web scraping and database maintenance.
Allows the system to automatically fetch and update data without user interaction.

Responsibilities:

- Triggers web scraping operations at scheduled intervals
- Interacts with `WebScraperService` to collect external data
- Uses `PetService` to process/store scraped data
- Calls `DatabaseBackupService` for backup-related operations

Key Features:

- Uses `@Scheduled` to run tasks automatically at defined times
- Uses `@Async` to run tasks outside the main thread (non-blocking)
- Enables continuous background processing without affecting API performance

Resources (Configuration Files)

Purpose: Environment-specific settings.
Profiles:

- Dev
- Test
- Production

Each profile has:

- Different database configs
- Different runtime behavior

Flyway (Database Migration)

Purpose: Manage database schema changes.

- Tracks migration versions
- Automatically updates schema
- Runs scripts in order

Testing

Purpose: Validate application behavior.

- Uses containerized MySQL database
- Separate configuration from dev/prod
- Includes:
  - Test data setup
  - Isolated environment

File: `TestcontainersConfiguration.java`

Layer: Testing – Configuration

Purpose: Configures a containerized MySQL database instance for integration testing using Testcontainers.

Responsibilities:

- Provides a MySQL container for the test environment
- Integrates containerized database with Spring Boot test context
- Ensures isolated and consistent database state for tests

Key Features:

- Uses `@TestConfiguration` to define test-specific beans
- Configures MySQL container via Testcontainers
- Uses `@ServiceConnection` to automatically bind container to Spring datasource
- Pulls Docker image dynamically using `mysql:latest`

File: `TestDataConfig.java`

Layer: Testing – Data Seeding

Purpose: Seeds predefined test data into the application at startup to ensure consistent and repeatable integration test scenarios.

Responsibilities:

- Inserts test users into the system
- Populates adoption sites and associated pets

- Ensures test environment has meaningful, relational data

Key Features:

- Uses `CommandLineRunner` to execute data seeding at startup
- Seeds multiple users via `UserService`
- Creates and persists `AdoptionSite` and `Pet` entities
- Handles duplicate or constraint exceptions gracefully
- Establishes relationships between pets and adoption sites

File: `PfaBase.java`

Layer: Testing – Base Class

Purpose: Abstract base class that provides a shared Spring Boot test context for all integration tests, optimizing performance through context reuse and centralized configuration.

Responsibilities:

- Initializes a shared Spring test context for all subclasses
- Integrates Testcontainers-based MySQL database
- Loads seeded test data before execution
- Provides common services and repositories for test classes

Key Features:

- Uses `@SpringBootTest` with random port for full-stack testing
- Imports test configurations (`TestcontainersConfiguration`, `TestDataConfig`)
- Activates `test` profile for environment-specific configuration
- Disables unnecessary WebDriver auto-configuration
- Enables Spring context caching to reduce container restarts
- Autowires commonly used services (JWTService, UserService, etc.)
- Supports token generation and authentication testing

File: `PfaContextTest.java`

Layer: Testing – Integration Test

Purpose: Validates that the Spring application context loads successfully with all required configurations and dependencies.

Responsibilities:

- Verifies application startup in a test environment
- Ensures all beans and configurations are correctly initialized

Key Features:

- Uses `@SpringBootTest` to load full application context
- Extends `PfaBase` to inherit shared test configuration
- Imports Testcontainers configuration for database support
- Provides a baseline sanity check for application stability

File: `ApiSecurityTest.java`

Layer: Testing – Security (Integration Test)

Purpose: Validates API security mechanisms, including authentication, authorization, and token handling, by simulating real HTTP requests against protected endpoints.

Responsibilities:

- Tests role-based access control for secured endpoints
- Verifies behavior of authenticated vs unauthenticated requests
- Ensures proper handling of malformed or missing tokens
- Confirms admin-only routes are restricted appropriately

Key Features:

- Uses `WebTestClient` to perform real HTTP requests against a running server
- Dynamically authenticates users and retrieves JWT tokens
- Validates HTTP status responses (401 Unauthorized, 403 Forbidden, 200 OK)
- Tests edge cases such as missing Bearer prefix and expired tokens
- Verifies admin privilege enforcement via role promotion

File: `JWTServiceTest.java`

Layer: Testing – Security (Integration Test)

Purpose: Ensures correctness, reliability, and security of JWT token generation, parsing, and validation across multiple scenarios.

Responsibilities:

- Validates token creation and structure
- Verifies expiration timing and enforcement
- Tests extraction of user identity from tokens and headers
- Ensures tokens are rejected when tampered with or misused

Key Features:

- Tests full JWT lifecycle (generation → validation → expiration)
- Verifies subject (email) extraction from token payload
- Validates behavior with malformed and improperly formatted tokens
- Ensures tokens cannot be reused across different users
- Confirms signature integrity and tamper detection
- Demonstrates key isolation across different `JWTService` instances

File: `PetServiceTest.java`

Layer: Testing – Service Layer (Integration Test)

Purpose: Validates the functionality of the PetService, including data retrieval, transformation, synchronization, and error handling.

Responsibilities:

- Tests retrieval of pet data from the database
- Verifies correct entity-to-DTO mapping
- Ensures proper handling of invalid queries
- Validates synchronization logic with external data sources

Key Features:

- Uses seeded data (Buddy, Luna, Max) for consistent test results
- Verifies multiple pet statuses (e.g., available, pending)
- Tests lookup by ID and exception handling for invalid IDs
- Confirms DTO fields are correctly populated
- Mocks `WebScraperService` to simulate external data ingestion
- Validates synchronization behavior, including duplicate handling and deactivation logic
- Ensures database consistency after sync operations

File: `UserServiceTest.java`

Layer: Testing – Security / Service Layer (Integration Test)

Purpose: Validates authentication logic, account management, and security-related behaviors within the UserService.

Responsibilities:

- Tests login validation and authentication failures
- Ensures duplicate account prevention
- Verifies secure password storage
- Validates user retrieval and role management

Key Features:

- Confirms invalid login attempts throw authentication exceptions
- Prevents duplicate account creation based on email
- Verifies passwords are stored as BCrypt hashes (never plaintext)
- Tests DTO safety by ensuring no sensitive fields (e.g., password) are exposed
- Validates user lookup by ID and email
- Confirms role promotion to admin is persisted
- Ensures email normalization (trimming and lowercase conversion)

File: `WebScraperServiceTest.java`

Layer: Testing – External Integration / Service Layer

Purpose: Validates the functionality and robustness of the WebScraperService, including HTML parsing, URL extraction, and dynamic data processing from external pet adoption sites.

Responsibilities:

- Tests extraction of URLs from HTML content
- Validates detection of supported scraping sources (e.g., ShelterLuv, PetFinder)
- Ensures correct parsing and transformation of scraped data
- Verifies builder pattern behavior for constructing pet data

Key Features:

- Uses Mockito to simulate Selenium WebDriver interactions
- Uses mocked HTML to replicate real-world scraping scenarios

- Validates extraction of anchor and iframe URLs
- Tests behavior with empty or malformed HTML inputs
- Confirms correct identification of scraping strategies based on URL patterns
- Verifies builder pattern resets internal state after object construction
- Ensures defensive copying to prevent unintended data mutation
- Tests data normalization (e.g., gender truncation, age conversion to weeks)
- Uses parameterized tests for multiple input scenarios

File: `TestPfaApplication.java`

Layer: Testing – Application Bootstrap

Purpose:
Provides a specialized entry point for launching the application in a test environment with Testcontainers-enabled configuration.

Responsibilities:

- Bootstraps the Spring Boot application for testing scenarios
- Injects test-specific configurations into the application context
- Ensures Testcontainers setup is applied during startup

Key Features:

- Uses `SpringApplication.from(...)` to reuse the main application configuration
- Extends the base `PfaApplication` while overriding configuration for testing
- Integrates `TestcontainersConfiguration` for containerized database support